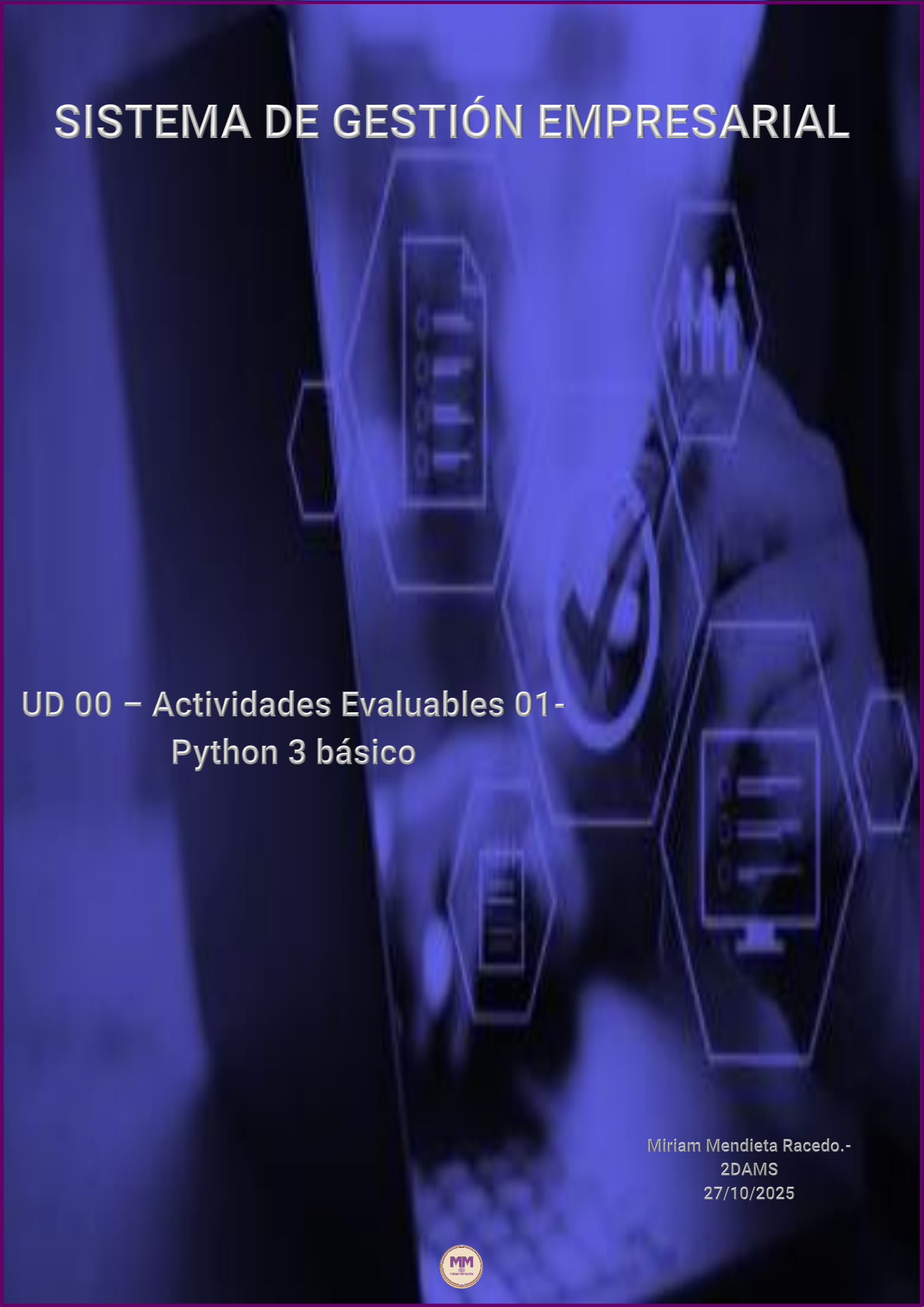
SISTEMA DE GESTIÓN EMPRESARIAL



UD 00 – Actividades Evaluables 01- Python 3 básico

Miriam Mendieta Racedo.-
2DAMS
27/10/2025

ÍNDICE

INTRODUCCIÓN **1**

ACTIVIDAD 01 2
ACTIVIDAD 02 10
ACTIVIDAD 03 13
ACTIVIDAD 04 15
ACTIVIDAD 05 18
ACTIVIDAD 06 24
ACTIVIDAD 07 26

BIBLIOGRAFIA **28**

INTRODUCCIÓN

Este trabajo reúne una serie de actividades prácticas relacionadas con la programación en **Python 3. Básico**.

El objetivo es consolidar conceptos fundamentales como la manipulación de listas, el uso de funciones, la gestión de datos con hash y la definición de clases. También se incluyen casos de uso reales como la gestión de usuarios y contraseñas, y la creación de instancias con atributos aleatorios, poniendo en práctica los conocimientos adquiridos.

“Python es indiscutiblemente uno de los lenguajes de programación más populares y versátiles del mundo”.

Su sintaxis clara y sencilla lo convierte en una excelente opción tanto para principiantes como para profesionales. Es un lenguaje interpretado, de alto nivel y multiparadigma, lo que significa que permite programar de forma estructurada, orientada a objetos o funcional. Sus aplicaciones son vastas y cubren áreas como el desarrollo web, la inteligencia artificial, el análisis de datos, la automatización de tareas, el desarrollo de videojuegos y la educación. La combinación de una comunidad de usuarios muy activa y la gran cantidad de bibliotecas disponibles lo establecen como un herramienta poderosa y eficiente para resolver problemas del mundo real.

Asimismo, Python se ha consolidado como una herramienta esencial en el entorno empresarial gracias a su capacidad para automatizar tareas, procesar grandes volúmenes de datos y desenrollar soluciones ágiles que optimizan la toma de decisiones. Por todo ello, se convierte en un motor clave de la transformación digital, impulsando la eficiencia y la innovación en organizaciones modernas de todos los sectores.

ACTIVIDAD 01

Prepara con un ejemplo donde explicas cómo hacer en Python 3:

● Clonar una lista.

Clonar una lista significa crear una copia completamente nueva de una lista existente. Esta copia tiene el mismo contenido, pero se encuentra en otra parte de la memoria, por lo que se puede modificar sin afectar a la lista original.

➢ ***Método de Slicing ([:])***
Método común y "pythónico". Crea una copia superficial de la lista completa al seleccionar todos los elementos desde el principio hasta el final.

```
original = [1, 2, 3, 4]
clonada = original[:]  # Copia por slicing

# se imprimen los  resultados
print(f"Original: {original}")   # Imprime la lista original.
print("Lista clonada:", clonada) # Imprime la lista clonada inicial.
print(".....")

# Se modifica clonada para probar la independencia
clonada.append(40)# Modifica solo la lista 'clonada' al añadir el elemento 40.
print(f"Original: {original}")#Imprime 'original'. Demuestra que NO fue modificada.
print(f"Lista clonada modificada: {clonada}")#Imprime 'clonada' y se muestra el nuevo elemento.

Original: [1, 2, 3, 4]
Lista clonada: [1, 2, 3, 4]
.....
Original: [1, 2, 3, 4]
Lista clonada modificada: [1, 2, 3, 4, 40]
```

➢ ***Método .copy()***
crea una copia superficial (shallow copy) de una lista. Se genera una nueva lista con los mismos elementos. Pero si alguno de esos elementos es una estructura mutable (como otra lista), no se copia, sino que se referencia.

```
#Método .copy()----------------------------------------------

lista_original = ['a', 'b', 'c']

# Clonación usando el método .copy()
lista_clonada = lista_original.copy()

#Modificar clonada
lista_clonada[0] = 'z'

# se imprimen resultados
print(f"Lista Original: {lista_original}")
print(f"Lista Clonada: {lista_clonada}")

Lista Original: ['a', 'b', 'c']
Lista Clonada: ['z', 'b', 'c']
```

- Método list() (constructor)
  Se puede pasar la lista original a la función constructora list() para crear una nueva lista a partir de los elementos de la original.

```
lista_original = [5, 15, 25]

#Clonación usando la función list()
lista_constructor = list(lista_original)#Se usa la función constructora list() para crear una nueva lista a partir de 'lista_original'.

#Verificación
lista_constructor.pop()#Elimina el último elemento (25) de 'lista_constructor'.

#se imprimen resultados
print(f"Lista Original: {lista_original}")
print(f"Lista por Constructor: {lista_constructor}")

Lista Original: [5, 15, 25]
Lista por Constructor: [5, 15]
```

Estos tres métodos estándar garantizan que la copia de una lista sea **independiente** de la original. Esto evita el error común de que ambas variables apunten a la misma referencia en memoria:

original = clonada → se modifican ambas

- Copias superficiales en listas anidadas

```
lista_original = [1, 2, [3, 4]]#lista anidada: contiene tres elementos: dos enteros (1, 2) y una lista interna [3, 4].
lista_copia = lista_original.copy   #Se realiza una copia superficial usando .copy() para duplicar lista_original. NO copia la sublista. En su lugar, copia la referencia (la dirección de memoria) donde se encuentra esa sublista.

lista_copia[2][0] = 999 #se accede al primer elemento de la sublista ([3, 4]) dentro de lista_copia y lo cambiamos a 999.

print("Original:", lista_original)  # [1, 2, [999, 4]]
print("Copia:", lista_copia)        # [1, 2, [999, 4]]

Original: [1, 2, [999, 4]]
Copia: [1, 2, [999, 4]]
```

La sublista [3, 4] es referenciada en ambas listas y por lo tanto, el cambio afecta también a lista_original.

- ¿Cuál es la diferencia en Python entre "shallow copy" y "deep copy"?

  - **Shallow copy**: es útil cuando los objetos no tienen estructuras anidadas., ya que copia el contenedor (la lista principal) pero solo las referencias a los objetos internos (anidados). Si modificas un objeto mutable (ej. una sublista) en la copia, el original también se verá afectado porque ambos apuntan al mismo objeto interno.

    list_original[:]
    list(list_original)

  - **Deep copy**: ideal para evitar efectos colaterales en estructuras complejas, como listas de listas o diccionarios anidados. Copia el contenedor y, recursivamente, copia todos los objetos anidados, creando una copia completamente independiente. Modificar cualquier objeto anidado en la copia no afectará al original.

    copy.deepcopy(list_original)

```python
import copy

# 1. Lista original con una lista anidada
lista_original = [1, 2, [3, 4]]
print(f"Original: {lista_original} (ID: {id(lista_original)})")

# ----------------------------------------------
#  Copia Profunda
# ----------------------------------------------
# Creamos una copia completamente independiente.
lista_profunda = copy.deepcopy(lista_original)
print(f"Profunda: {lista_profunda} (ID: {id(lista_profunda)})")

# Modificamos el elemento ANIDADO en la copia profunda
lista_profunda[2][0] = 999  # Cambiamos el '3' por '999' en la sublista

print("\n--- Después de modificar la Copia Profunda ---")
print(f"Profunda: {lista_profunda}")
print(f"Original: {lista_original}")

print("\n" + "="*50 + "\n")

# ----------------------------------------------
# Copia Superficial (Para ver la diferencia)
# ----------------------------------------------
# Usamos un slice para una copia superficial
lista_superficial = lista_original[:]
print(f"Superficial: {lista_superficial}")

# Modificamos el elemento ANIDADO en la copia superficial
lista_superficial[2][0] = 777  # Cambiamos el '3' por '777' en la sublista

print("\n--- Después de modificar la Copia Superficial ---")
print(f"Superficial: {lista_superficial}")
print(f"Original: {lista_original}")
print()
```

```
Original: [1, 2, [3, 4]] (ID: 1978742628544)
Profunda: [1, 2, [3, 4]] (ID: 1978745011712)

--- Después de modificar la Copia Profunda ---
Profunda: [1, 2, [999, 4]]
Original: [1, 2, [3, 4]]

==================================================

Superficial: [1, 2, [3, 4]]

--- Después de modificar la Copia Superficial ---
Superficial: [1, 2, [777, 4]]
Original: [1, 2, [777, 4]]
```

- Añadir un elemento a una lista.

  - El método **.append()** añade el nuevo elemento al final de la lista.

```python
# lista inicial
mi_lista = ["manzana", "banana", "cereza"]

# se imprime los elementos y su longitug
print(f"Lista original: {mi_lista}")
print(f"Elementos: {len(mi_lista)}") #longitud de la lista

nuevo_elemento = "uva"
mi_lista.append(nuevo_elemento) #se añade un nuevo elemento al final y se imprime
print(mi_lista)
print(f"Elementos: {len(mi_lista)}") #longitud de la lista despúes de append()
```

```
Lista original: ['manzana', 'banana', 'cereza']
Elementos: 3
['manzana', 'banana', 'cereza', 'uva']
Elementos: 4
```

- Método insert() → se añade en una posición específica.
  Si se necesita añadir el elemento **en medio** de la lista o al principio se pude usar este método.

```
mi_otra_lista = ["rojo", "verde", "azul"]

# Añade "amarillo" en la posición 1 (desplazando "verde" y "azul")
mi_otra_lista.insert(1, "amarillo")# .insert(índice, elemento)

print(f"\nResultado: {mi_otra_lista}") # Resultado: ['rojo', 'amarillo', 'verde', 'azul']
```

- Método extend() → Añade varios elementos
  Se puede utilizar para añadir todos los elementos de un iterable (como otra lista, una tupla o un rango) al final de una lista existente.

```
# Lista actual (la que será extendida)
lista_principal = [10, 20, 30]
print(f"Lista principal (antes): {lista_principal}")

#  Iterable con los elementos a añadir)
elementos_a_añadir = [40, 50, 60]

lista_principal.extend(elementos_a_añadir)# fusiona los contenidos de un iterable dentro de la lista actual.
print(elementos_a_añadir)

print(f"Lista principal (después): {lista_principal}")

Lista principal (antes): [10, 20, 30]
[40, 50, 60]
Lista principal (después): [10, 20, 30, 40, 50, 60]
```

- operador + → Combina listas

```
# listas que queremos combinar
lista_animales = ["perro", "gato"]
lista_aves = ["loro", "canario"]
lista_peces = ["pez dorado"]
#Se usa el operador + para combinarlas, se  crea una nueva lista 'lista_completa'
```

```
lista_completa = lista_animales + lista_aves + lista_peces
#  resultado
print(f"Lista 1 (Animales): {lista_animales}")
print(f"Lista 2 (Aves): {lista_aves}")
print(f"Lista 3 (Peces): {lista_peces}")

print("\n--- Después de usar el operador + ---")
print(f"Lista Combinada: {lista_completa}")
print()
mi_lista = ['A']
nueva_lista = mi_lista + ['B', 'C']
print(nueva_lista)
Lista 1 (Animales): ['perro', 'gato']
Lista 2 (Aves): ['loro', 'canario']
Lista 3 (Peces): ['pez dorado']

--- Después de usar el operador + ---
Lista Combinada: ['perro', 'gato', 'loro', 'canario', 'pez dorado']

['A', 'B', 'C']
```

- Quitar un elemento a una lista.

```
# se crea una lista inicial
mi_lista = [1, 2, 3, 4, 5]
print(mi_lista)

# se mueve e elemento
elemento = mi_lista.pop(2) # Esto elimina el elemento en la posición 2
# mi_lista ahora es [1, 2, 4, 5]

# se añade el elemento a otra lista
otra_lista = [ ]
otra_lista.append(elemento)  # Añadimos el elemento 3 a otra_lista
# otra_lista ahora es [3]
print(mi_lista)
print(otra_lista,"\n")
```

● Crear una nueva lista con los 4 últimos elementos de una lista.

```
lista_original = [10, 20, 30, 40, 50, 60, 70]

# Usamos slicing para obtener los últimos 4 elementos
cuatro_ultimos = lista_original[-4:]
# Se usa el 'slicing' (rebanado) para obtener los últimos N elementos.
# El índice negativo '-4' indica el cuarto elemento contando desde el final.
# Los dos puntos ':' al final indican 'hasta el final de la lista'.

print(cuatro_ultimos,"\n")
```

● Convertir las palabras de una cadena (separadas por espacio) a una lista.

```
# se define una cadena de texto donde las palabras están separadas por espacios

texto = "Python es muy útil para programar"

# El método 'split()' de una cadena divide la cadena en una lista de subcadenas.
# Por defecto, 'split()' usa el espacio en blanco como separador.
lista_palabras = texto.split()

print(lista_palabras,"\n")
# Resultado esperado: ['Python', 'es', 'muy', 'útil', 'para', 'programar']
```

● Comentarios con una línea.

```
# Para hacer un comentario de una sola línea se utiliza el símbolo #.
   # Los comentarios son ignorados por el intérprete de Python y se utilizan para facilitar la comprensión del código.
```

● Comentarios multilínea.

```
"""
  Comentario multilínea (o docstring si está al inicio de una función/clase):
  Este bloque de texto abarca varias líneas y se usa para explicaciones
  más detalladas, como la finalidad general de un archivo o un bloque de código extenso.
  Se utiliza comillas dobles o simples(" o ' )
"""
```

ACTIVIDAD 02

En Python 3 los tipos simples pasan por valor y los compuestos por referencia.

Crea un ejemplo con 3 funciones que:

● Recibe 2 números y devuelva la suma.

```
def sumar(num1, num2):#Toma dos números como argumentos.
    """
    Funcion que suma dos números.

    Args:
        num1 (int, float): El primer número.
        num2 (int, float): El segundo número.

    Retuns:
        int, float: La suma de num1 y num2.
    """
return num1 + num2 #Devuelve la suma de los dos números.
#paso por valor, ya que los números son tipos simples.
```

- Recibe una lista y modifique esa misma lista (referencia) doblando los valores de todos los elementos. No tiene que devolver nada.

```
def doblar_valores(lista):
    """
    Función que dobla cada valor de la lista y modifica la lista original.

    Args:
        lista (list): La lista a modificar. Cada elemento debe ser un número.

    Returns:
        None

    Modifica:
        La lista original se actualiza con los valores doblados.
    """
    for i in range(len(lista)):
        lista[i] *= 2  # Multiplica cada elemento de la lista por 2
    #la lista (tipos compuestos) se pasan por referencia, y los cambios se reflejan en la lista original.
```

- Recibe una lista y devuelva una copia de la lista misma lista (referencia) doblando los valores de todos los elementos. La lista original no debe modificarse.

```
def duplicar_valores(lista):
    """
    Función que crea una nueva lista con los valores de la lista original doblados.
    Args:
        lista (list): La lista cuyos valores se duplicarán. Cada elemento debe ser un número.
    Retuns:
        list: Una nueva lista con los valores doblados.
    """
    nueva_lista = [ ]  # Inicializa una nueva lista vacía
    for elemento in lista:# Itera sobre la lista original.
        nueva_lista.append(elemento * 2)  # multiplica cada elemento por 2 y los añade a la nueva lista
    return nueva_lista  # Devuelve la nueva lista
```

```
# Programa principal
"""
Bloque que permite ejecutar código de prueba o ejemplos solo cuando el archivo se ejecuta directamente, evitando su ejecución al importar como módulo.
"""
if __name__ == "__main__":
    # Sumar dos números
    resultado_suma = sumar(3, 5)
    print("La suma de 3 y 5 es:", resultado_suma)

    # Modificar la lista original
    lista_original = [1, 2, 3, 4]
    print("Lista antes de doblar valores:", lista_original)
    doblar_valores(lista_original)
    print("Lista después de doblar valores:", lista_original)

    # Crear una nueva lista con valores doblados
    lista_a_doblar = [1, 2, 3, 4]
    nueva_lista = duplicar_valores(lista_a_doblar)
    print("Lista original:", lista_a_doblar)
    print("Nueva lista con valores doblados:", nueva_lista)

La suma de 3 y 5 es: 8
Lista antes de doblar valores: [1, 2, 3, 4]
Lista después de doblar valores: [2, 4, 6, 8]
Lista original: [1, 2, 3, 4]
Nueva lista con valores doblados: [2, 4, 6, 8]
```

ACTIVIDAD 03

Partiendo de un contexto en el que queremos almacenar un usuario y su contraseña. Haz un ejemplo

que explica cómo se haría:

- Utilizando una lista.
- Utilizando un diccionario.

Al llenarse, las contraseñas deben pasarse a un formato Hash (por ejemplo SHA https://recursos-python.com/guias-y-manuales/hashlib-md5-sha/). El ejemplo debe llenar la lista con 5 usuarios/contraseña y hacer dos consultas.

```python
import hashlib

# Función para hashear contraseñas
def hash_password(password):
    """
    Convierte una contraseña en un hash SHA-256.

    Args:
        password (str): La contraseña ingresada.

    Returns:
        str: La representación hexadecimal del hash SHA-256.
    """

    # 'password' es una cadena de texto (str), la convertimos a bytes
    password_bytes = password.encode()  #a función encode() convierte la cadena de texto de la contraseña en un objeto de tipo bytes.se usa la codificación UTF-8 por defecto.

    # se crea  un objeto hash SHA-256
    sha256_hash = hashlib.sha256(password_bytes)

    # se convierte el hash a formato hexadecimal
    hash_hex = sha256_hash.hexdigest()

    # se retorna el hash en formato hexadecimal
    return hash_hex #hashlib.sha256(password.encode()).hexdigest()
# ----------------------------
```

```
#      Lista de tuplas
# ----------------------------

usuarios = [ ]

# Llenar la lista con 5 usuarios y sus contraseñas (en hash)
usuarios.append(("usuario1", hash_password("contraseña1")))
usuarios.append(("usuario2", hash_password("contraseña2")))
usuarios.append(("usuario3", hash_password("contraseña3")))
usuarios.append(("usuario4", hash_password("contraseña4")))
usuarios.append(("usuario5", hash_password("contraseña5")))

# Función para consultar en la lista
def consultar_usuario_lista(nombre_usuario):
    for usuario, password_hash in usuarios:
        if usuario == nombre_usuario:
            return f"Usuario: {usuario}, Contraseña (hash): {password_hash}"
    return "Usuario no encontrado."

# Consultas
print(consultar_usuario_lista("usuario2"))
print(consultar_usuario_lista("usuario5"))
print(consultar_usuario_lista("usuario8"))  # No existe


# ----------------------------
#      Diccionario
# ----------------------------

usuarios_dict = {
    "usuario11": hash_password("contraseña11"),
    "usuario22": hash_password("contraseña22"),
    "usuario33": hash_password("contraseña33"),
    "usuario44": hash_password("contraseña44"),
    "usuario55": hash_password("contraseña55")
}

# Función para consultar en el diccionario
```

```
def consultar_usuario_dict(nombre_usuario):
    if nombre_usuario in usuarios_dict:
        return f"Usuario: {nombre_usuario}, Contraseña (hash): {usuarios_dict[nombre_usuario]}"
    return "Usuario no encontrado."

# Consultas
print(consultar_usuario_dict("usuario33"))  # Existe
print(consultar_usuario_dict("usuario11"))  # Existe
print(consultar_usuario_dict("usuario99"))  # No existe
```

```
Usuario: usuario2, Contraseña (hash):
3fb14b60d144c0c85deb4f01d8e8c95c46d301fe18d0cf3575931e488b0fda6f
Usuario: usuario5, Contraseña (hash):
c9391add1c2695e903be0a61ae93b225ab7c79189dad93165ccf35ac52215f2b
Usuario no encontrado.
Usuario: usuario33, Contraseña (hash):
2317bf9a7cee579c6ae74a608870f74fae2848abad1ca0ff2662e8a7e0b5c824
Usuario: usuario11, Contraseña (hash):
e2a29191041b52b77e034585c3f973d4e20187d9da4e16e99369fb7b527b2ceb
Usuario no encontrado.
```

ACTIVIDAD 04

Explica con ejemplos cómo funcionan los operadores "is", "not", "in" en Python 3.

```
# ----------------------------
#        operador "is"
# ----------------------------

'''
El operador is se utiliza para comparar la identidad de dos objetos en Python.
Verifica si ambas referencias apuntan al mismo objeto en memoria, no solo si son equivalentes.
'''
print("operador 'is' ")
a = [1, 2, 3]
b = a   # b es una referencia al mismo objeto que a
```

```
c = list(a)  # c es una copia de a, no es el mismo objeto

print(a is b)  # True, porque b se refiere al mismo objeto que a
print(a is c)  # False, porque c es un nuevo objeto
print()

# ----------------------------
#        operador "not"
# ----------------------------
'''
El operador not es un operador lógico que invierte el valor de verdad de una expresión. Si la expresión es
True, not la convierte en False y viceversa.
'''
print("\noperador 'not' ")
activo = False
if not activo:
    print("El usuario no está activo.")  # Se imprime porque activo es False

es_adulto = True
es_estudiante = False

print(not es_adulto)   # False, porque es_adulto es True
print(not es_estudiante)  # True, porque es_estudiante es False

x = 5
print(not x > 10)  # True → x no es mayor que 10
print

# ----------------------------
#        operador "in"
# ----------------------------

'''
El operador in verifica si un elemento está presente en una secuencia (como listas, tuplas, cadenas o con-
juntos).
Devuelve True si el elemento se encuentra en la secuencia; de lo contrario, devuelve False
'''
print("\noperador 'in' ")
```

```
frutas = ["manzana", "banana", "cereza"]
print("banana" in frutas)  # True, porque "banana" está en la lista
print("naranja" in frutas)  # False, porque "naranja" no está en la lista

mensaje = "Hola Miriam"
print("Miriam" in mensaje)  # True

colores = ["rojo", "azul", "verde"]
print("amarillo" not in colores)  # True

usuarios = {"ana": "123", "juan": "456"}
print("ana" in usuarios)  # True
print("456" in usuarios)  # False porque "456" NO es una clave, es un valor asociado a la clave "juan".
Print( )


operador 'is'
True
False

operador 'not'
El usuario no está activo.
False
True
True

operador 'in'
True
False
True
True
True
False
```

ACTIVIDAD 05

Pone un ejemplo de cómo pasar varios parámetros desde consola a un programa Python 3.

Pone un ejemplo de cómo hacer "sobrecarga de funciones" (funciones que pueden recibir varios números de parámetros), incluyendo el caso de que el número de parámetros no sea definido.

Usando *args (número indefinido de argumentos)

```
def mostrar_tareas(*tareas): #El asterisco (*) indica que aceptará un número variable de argumentos posicionales,que se agruparán en una tupla llamada 'tareas'.
    '''
        Función que muestra una lista de tareas.
        Args:
            *tareas: Número indefinido de tareas como argumentos.
        Returns:
            None
    '''
    for i, tarea in enumerate(tareas, 1):#Inicia un bucle 'for' para iterar sobre la tupla 'tareas'.
# La función 'enumerate' asigna un índice a cada elemento.
# El argumento '1' indica que la numeración debe comenzar desde 1 (no desde 0).
# 'i' recibirá el número de tarea (1, 2, 3...), y 'tarea' recibirá el contenido de la tarea.
        print(f"{i}. {tarea}")#Se usa una f-string para formatear la salida

mostrar_tareas("Diseñar UI", "Depurar código", "Documentar fichas")
# Llama a la función 'mostrar_tareas' con tres argumentos.
# Estos tres strings se recogen dentro de la tupla 'tareas' de la función.

print()#salto de linea

1. Diseñar UI
2. Depurar código
3. Documentar fichas
```

```
def calcular_suma(*args):
    '''
        Suma cualquier número de argumentos numéricos.
        Args:
            *args: tupla con todos los valores pasados(Número indefinido).
```

```
    Returns:
        float: La suma de todos los argumentos.
    '''
    if not args:
        return 0 # Retorna 0 si no hay argumentos

    suma = sum(args)
    # calcula la suma de todos los elementos dentro de la tupla 'args'
    # utilizando la función incorporada 'sum()' de Python, y guarda el resultado en la variable 'suma'.

    print(f"Números a sumar: {args}")
    # Imprime por pantalla la tupla completa de argumentos ('args') que están siendo sumados.
    # f-string para incluir el valor de la variable 'args' en el mensaje.

    return suma
    # Retorna el valor calculado de la suma de los argumentos, finalizando la ejecución de la función.

print()

# Ejemplos de uso
# 1. Llamada sin argumentos
print(f"Resultado: {calcular_suma()}")
# Salida: Números a sumar: () / Resultado: 0

# 2. Llamada con 3 argumentos
print(f"Resultado: {calcular_suma(10, 20, 30)}")
# Salida: Números a sumar: (10, 20, 30) / Resultado: 60

# 3. Llamada con 5 argumentos
print(f"Resultado: {calcular_suma(1, 2, 3, 4, 5)}")
# Salida: Números a sumar: (1, 2, 3, 4, 5) / Resultado: 15
print()
```

- *args permite que una función reciba muchos argumentos posicionales sin definir cuántos(representa entrada abierta y adaptable).
- Internamente, se convierte en una tupla que puedes recorrer o procesar.
- Es útil cuando quieres que tu función sea flexible y reutilizable.
- Puedes usar cualquier nombre en lugar de args, pero el asterisco * es obligatorio.
- Se usa cuando los argumentos no tienen nombre, solo posición.
- Es perfecto para funciones como sumar(1, 2, 3, 4) sin limitar el número de entradas.
- Puedes combinarlo con **kwargs para aceptar también argumentos con nombre.

****

Usando **kwargs (argumentos con nombre)

```
def mostrar_info(**datos):# Define una función llamada 'mostrar_info'.
# El doble asterisco (**) indica que aceptará un número variable de argumentos de palabra clave (key-value pairs), los cuales serán recolectados en un diccionario llamado 'datos'.

   for clave, valor in datos.items(): # para iterar sobre los pares clave-valor del diccionario 'datos'.
# El método '.items()' devuelve pares (clave, valor) para cada elemento del diccionario.
# 'clave' tomará el nombre del argumento (ej: "nombre", "proyecto"), y 'valor' tomará su valor (ej: "Miriam", "DAM").

      print(f"{clave}: {valor}")# Imprime cada par clave-valor en un formato legible. Se utiliza una f-string para formatear la salida, como "nombre: Miriam".

mostrar_info(nombre="Miriam", proyecto="DAM", año=2025) # Llama a la función 'mostrar_info' y le pasa tres argumentos de palabra clave.
                                    # Estos tres pares se combinan en el diccionario 'datos' dentro de la función.
print()

nombre: Miriam
proyecto: DAM
año: 2025
```

```
def configurar_objeto(nombre, **kwargs):
    """
        Configura un objeto con propiedades adicionales(nombre requerido y opciones indefinidas).
        'kwargs' es un diccionario de opciones clave-valor.
        Args:
            nombre (str): Nombre del objeto a configurar.
            **kwargs: Propiedades adicionales como pares clave-valor.
        Returns:
            None
            Configura un objeto con un n.
        """

    print(f"Configurando el objeto: {nombre}") # Muestra el nombre del objeto que se está configurando.

    for clave, valor in kwargs.items():
        print(f"  -> {clave}: {valor}")# Itera sobre cada par clave-valor en kwargs y los imprime con formato. Esto permite visualizar todas las propiedades adicionales del objeto.

    if 'color' not in kwargs:
        print("Advertencia: No se especificó el color.") # Verifica si no se ha proporcionado la clave 'color' en kwargs. Si falta, muestra una advertencia para alertar al usuario.

# 1. Llamada con 3 parámetros nombrados (incluye 'color')
configurar_objeto("Ventana Principal", ancho=800, alto=600, color="azul")
print()
# 2. Llamada con 2 parámetros nombrados (sin 'color')
configurar_objeto("Botón OK", texto="Aceptar", estilo="plano")
print()

Configurando el objeto: Ventana Principal
  -> ancho: 800
  -> alto: 600
  -> color: azul

Configurando el objeto: Botón OK
  -> texto: Aceptar
  -> estilo: plano
Advertencia: No se especificó el color.
```

```
def ficha_tecnica(**kwargs):
    '''
        Genera una ficha técnica a partir de propiedades dinámicas.
        Args:
            **kwargs: Propiedades como pares clave-valor.
        Returns:
            str: Ficha técnica formateada.
    '''
    ficha = "Ficha Técnica\n"
    for clave, valor in kwargs.items():
        ficha += f"{clave}: {valor}\n"
    return ficha

print(ficha_tecnica(nombre="Servidor", sistema="Ubuntu", versión="24.04"))
print()

Ficha Técnica
nombre: Servidor
sistema: Ubuntu
versión: 24.04
```

**kwargs permite que una función reciba un número indefinido de argumentos nombrados (clave=valor).Internamente, kwargs se convierte en un diccionario.
Es útil cuando no se sabe cuántos parámetros opcionales se van a pasar, o se quiere permitir configuraciones flexibles.
usar **kwargs cuando:

- ✓ Se quiere permitir parámetros opcionales con nombre.
- ✓ Se está creando funciones reutilizables y adaptables.
- ✓ Se necesita pasar configuraciones sin definir cada parámetro explícitamente.
- ✓ se envolve funciones (por ejemplo, en decoradores).

Combinando *args y **kwargs

```
def procesar(*args, **kwargs): # Imprime los argumentos posicionales (almacenados en la tupla 'args').
    '''
    Procesa argumentos posicionales y nombrados.
    Args:
        *args: Argumentos posicionales.
        **kwargs: Argumentos nombrados.
    Returns:
        None
    '''
    print("Posicionales:", args)
    print("Nombrados:", kwargs)

# Llama a la función 'procesar'.
# 1, 2 y 3 se pasan como argumentos posicionales.
# modo="debug" y verbose=True se pasan como argumentos de palabra clave.
procesar(1, 2, 3, modo="debug", verbose=True)

Posicionales: (1, 2, 3)
Nombrados: {'modo': 'debug', 'verbose': True}
```

```
def procesar_entrada(*args, **kwargs): # Imprime la tupla que contiene los argumentos posicionales.
    print("Posicionales:", args)
    # Imprime el diccionario que contiene los argumentos nombrados/de palabra clave.
    print("Nombrados:", kwargs)

# Llama a la función 'procesar_entrada'.
# "inicio" y "carga" se pasan como argumentos posicionales.
# modo="debug" y verbose=True se pasan como argumentos de palabra clave.
procesar_entrada("inicio", "carga", modo="debug", verbose=True)
print()

Posicionales: ('inicio', 'carga')
Nombrados: {'modo': 'debug', 'verbose': True}
```

Parámetros opcionales

```
def saludar(nombre, saludo="Hola"):# Imprime una cadena formateada (f-string) que combina el saludo y el nombre.
    '''
        Saluda a una persona con un saludo personalizado.
        Args:
            nombre (str): Nombre de la persona a saludar.
            saludo (str, optional): Saludo a utilizar. Por defecto es "Hola".
        Returns:
            None
    '''
    print(f"{saludo}, {nombre}!")

# Primera llamada a la función:
# Se proporciona solo el argumento 'nombre' ("Miriam").
# El argumento 'saludo' toma su valor por defecto: "Hola".
saludar("Miriam")

# Segunda llamada a la función:
# Se proporcionan ambos argumentos: 'nombre' ("Miriam") y 'saludo' ("Buenos días").
# El valor por defecto de 'saludo' es sobrescrito por "Buenos días".
saludar("Miriam", "Buenos días")
print()

Hola, Miriam!
Buenos días, Miriam!
```

ACTIVIDAD 06

Crea un listado en el cual cada elemento de esa lista sea una lista con dos valores: medida y peso. Utilizando https://docs.python.org/3/howto/sorting.html y las “key functions”, hacer que esta lista se ordene por mayor altura y en caso de igualdad, por menor peso.

Explica en comentarios que es realmente la “key function”. Pista: en la ayuda dicen

"The value of the key parameter should be a function (or other callable) that takes a single argument and returns a key to use for sorting purposes. This technique is fast because the key function is called exactly once for each input record."

```
# ----------------------------
#       key functions
# ----------------------------
# lista de medidas y pesos
datos = [
    [170, 65],
    [180, 70],
    [170, 60],
    [160, 55],
    [180, 75],
    [170, 65]
]
#1.- Función definida-------------------------------------------------------------
# Función que devuelve la clave de ordenación
def clave_ordenacion(par):
    altura = par[0]
    peso = par[1]
    return (-altura, peso)  # Altura descendente, peso ascendente
# se ordena  usando la función como key
datos_ordenados = sorted(datos, key=clave_ordenacion)

# se muestra el resultado
for altura, peso in datos_ordenados:
    print(f"Altura: {altura} cm, Peso: {peso} kg")

#2.- Función lambda---------------------------------------------------------------

# se ordena usando una función lambda como key
datos_ordenados = sorted(datos, key=lambda x: (-x[0], x[1]))

# se muestra el resultado
print("Lista ordenada por altura descendente y peso ascendente:")
for persona in datos_ordenados:
    print(f"Altura: {persona[0]} cm, Peso: {persona[1]} kg")
```

```
#3- Usar itemgetter del módulo operator----------------------------------
from operator import itemgetter

# se ordena primero por peso ascendente
datos.sort(key=itemgetter(1))

# Luego por altura descendente
datos.sort(key=itemgetter(0), reverse=True)

#  resultado
for altura, peso in datos:
    print(f"Altura: {altura} cm, Peso: {peso} kg")
'''
La key function  se utiliza para especificar los criterios según los cuales se realiza la ordenación de una lista. Cuando se llama al método sort() en Python, se puede proporcionar un argumento llamado key, que es una referencia a la key function.
La función debe tomar un solo argumento y devolver un valor que se utilizará para la comparación durante la ordenación.
Ejemplo en este caso: En el código anterior, la función key_function toma cada elemento de la lista que tiene dos valores: la medida  y el peso. Negamos la altura para que se ordene de mayor a menor.
Si hay empate en las alturas, Python utiliza el peso, que se ordenará de menor a mayor, automáticamente gracias a la naturaleza de la tupla que se devuelve.
'''
Lista ordenada por altura descendente y peso ascendente:
Altura: 180 cm, Peso: 70 kg
Altura: 180 cm, Peso: 75 kg
Altura: 170 cm, Peso: 60 kg
Altura: 170 cm, Peso: 65 kg
Altura: 170 cm, Peso: 65 kg
Altura: 160 cm, Peso: 55 kg
Altura: 180 cm, Peso: 70 kg
```

ACTIVIDAD 07

Define La clase Car en Python 3. La clase tendrá como atributos “matrícula” (numérica) y “color”.

Crea un método imprimir, y además dos métodos que quieras.

En segundo lugar, hacer que el programa pido un número “n” por teclado y se crean “n” instancias de la clase, donde cada instancia:

- Cada “matrícula” tendrá un número consecutivo desde 1 hasta “n”.
- El “color” será para cada instancia un color aleatorio obtenido de este listado [“red”, “white”, “*black”, “pink”, “blue”]

Finalmente, el programa tendrá que imprimir los valores de las 10 primeras instancias. En caso de que “n” sea menor que 10, solo imprimirá “n” instancias.

```python
import random  # Importa el módulo para generar colores aleatorios

# ---  clase Car ---
class Car:
    def __init__(self, matricula, color):
        self.matricula = matricula  # Atributo numérico único
        self.color = color          # Atributo de color aleatorio

    def imprimir(self):
        # Muestra los datos del coche
        print(f"Matrícula: {self.matricula}, Color: {self.color}")

    def arrancar(self):
        # Método adicional: simula el arranque
        print(f"Carro {self.matricula} está arrancando...")

    def detener(self):
        # Método adicional: simula la detención
        print(f"Carro {self.matricula} se ha detenido.")

# --- Lista de colores disponibles ---
colores_disponibles = ["red", "white", "*black", "pink", "blue"]

# --- Solicita al usuario cuántos coches crear ---
n = int(input("¿Cuántos coches quieres crear? "))  # Entrada por teclado

# --- Crea 'n' instancias de Car con matrícula secuencial y color aleatorio ---
flota = []  # Lista para almacenar los objetos Car

for i in range(1, n + 1):
```

```
    color_aleatorio = random.choice(colores_disponibles)  # Selección aleatoria
    coche = Car(matricula=i, color=color_aleatorio)      # Instancia de Car
    flota.append(coche)                                  # Añade a la lista

# --- Imprime los datos de los primeros 10 coches (o menos si n < 10) ---
print("\nFicha de los primeros coches:")
for coche in flota[:10]:  # Slicing para limitar a 10
    coche.imprimir()

¿Cuántos coches quieres crear? 15

Ficha de los primeros coches:
Matrícula: 1, Color: blue
Matrícula: 2, Color: pink
Matrícula: 3, Color: red
Matrícula: 4, Color: pink
Matrícula: 5, Color: white
Matrícula: 6, Color: red
Matrícula: 7, Color: blue
Matrícula: 8, Color: blue
Matrícula: 9, Color: red
Matrícula: 10, Color: blue
```

BIBLIOGRAFIA

- Apunts del Professor.
- Python para todos. Explorando la información con Python 3. Charles R. Severance
- https://docs.python.org/es/3/library/copy.html
- SHA https://recursospython.com/guias-y-manuales/hashlib-md5-sha/

- https://docs.python.org/3/howto/sorting.html